گزارش پروژه

# تحلیل سیگنالینگ BGP VPLS و استخراج MTU از فایلPacket Capture

**نام :** حامدربانی
**زبان برنامه‌نویسی:** Python
**ابزارهای اصلی:**

- Scapy
- pytest
- Git
- GitHub

## ۱- مقدمه:

در شبکه‌های کامپیوتری، بررسی ترافیک ثبت‌شده یکی از روش‌های مهم برای شناخت رفتار پروتکل‌ها و عیب‌یابی ارتباطات شبکه است. با استفاده از فایل‌های Packet Capture می‌توان پیام‌های ردوبدل‌شده بین تجهیزات را بررسی کرد و اطلاعات موردنیاز را از داخل آن‌ها استخراج کرد.

در این پروژه، یک فایل ضبط‌شده با فرمت PCAPNG بررسی شده تا پیام‌های مربوط به پروتکل BGP و سرویس VPLS شناسایی شوند. تمرکز اصلی پروژه روی استخراج اطلاعات سیگنالینگ VPLS و بررسی مقادیر MTU اعلام‌شده در پیام‌های BGP است.

برای انجام این کار، برنامه‌ای با زبان Python توسعه داده شده که اطلاعات موجود در فایل Capture را پردازش می‌کند و نتایج را در قالب یک گزارش JSON ذخیره می‌کند.

همچنین برای بررسی صحت بخش‌های مختلف برنامه، تست‌های خودکار نوشته شده و برای مدیریت نسخه‌های کد و آماده‌سازی پروژه جهت انتشار در GitHub از Git استفاده شده است.

-https://github.com/HamedRabbani/vpls-mtu-validation

## ۲- هدف پروژه:

هدف اصلی این پروژه، خودکارسازی بخشی از فرایند بررسی پیام‌های BGP مرتبط با VPLS است.

اهداف جزئی پروژه عبارت‌اند از:

- **خواندن فایل‌های شبکه با فرمت PCAP وPCAPNG :**

```
(Sina_Communications) PS D:\VPLS_Packet_Capture_Test> python scripts/inspect_bgp_mtu.py
Packets read: 4897
Directed TCP streams: 2
BGP messages extracted: 51
BGP UPDATE messages parsed: 26
Invalid or unparsed messages: 25
Report saved to: reports\bgp_vpls_mtu.json
{
  "bgp_updates": 26,
  "vpls_related_updates": 18,
  "advertised_mtu_values": [
    1500,
    9000
  ],
  "updates_with_parse_errors": 0
}
```

**شناسایی جریان‌های TCP مرتبط با ارتباطBGP :**

```
(Sina_Communications) PS D:\VPLS_Packet_Capture_Test> python scripts/inspect_bgp_mtu.py
Packets read: 4897
Directed TCP streams: 2
BGP messages extracted: 51
BGP UPDATE messages parsed: 26
Invalid or unparsed messages: 25
Report saved to: reports\bgp_vpls_mtu.json
{
  "bgp_updates": 26,
  "vpls_related_updates": 18,
  "advertised_mtu_values": [
    1500,
    9000
  ],
  "updates_with_parse_errors": 0
}
```

**استخراج پیام‌های BGP از جریان‌هایTCP :**

```
def extract_bgp_messages(stream: bytes) -> list[bytes]:
    """Extract complete BGP messages from a TCP stream."""

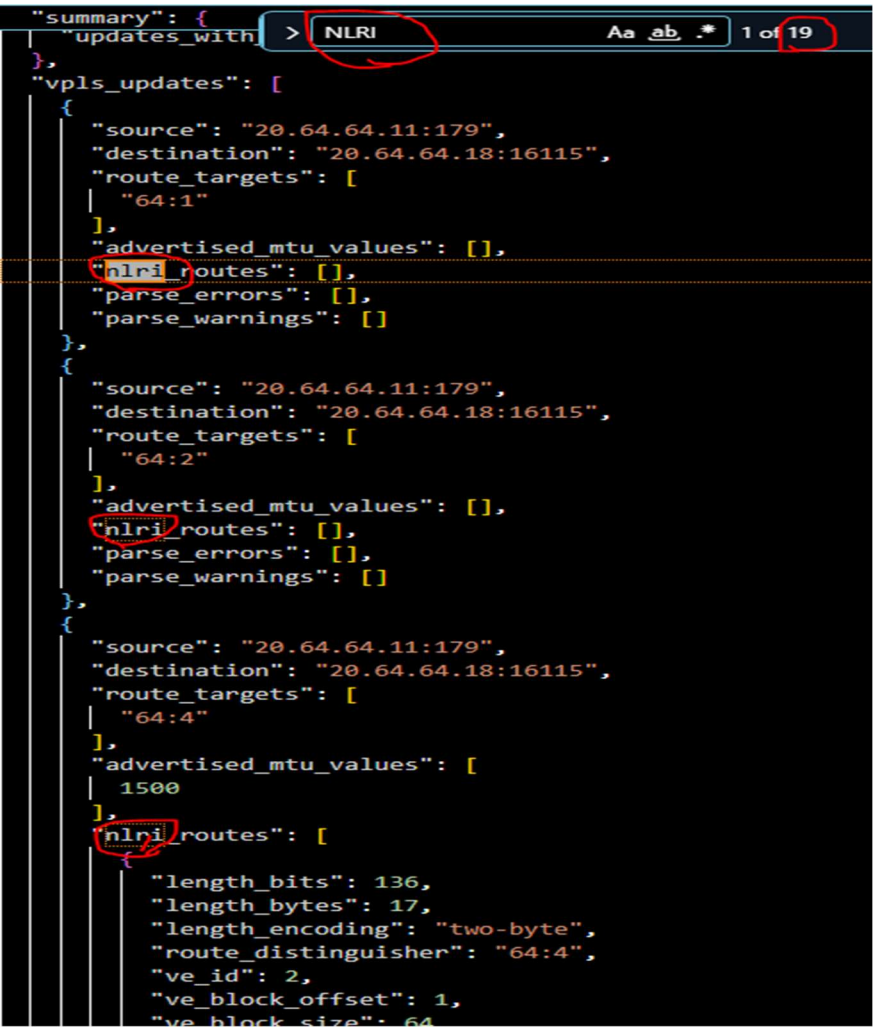
    messages = []
    offset = 0
    marker = b"\xff" * 16
```

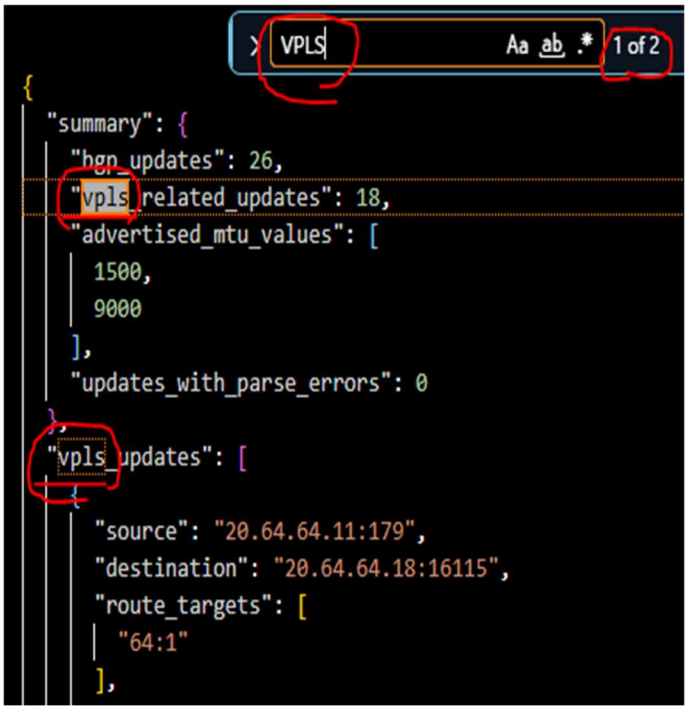
**شناسایی و پردازش پیام‌هایBGP UPDATE :**

```
(Sina_Communications) PS D:\VPLS_Packet_Capture_Test> python scripts/inspect_bgp_mtu.py
Packets read: 4897
Directed TCP streams: 2
BGP messages extracted: 51
BGP UPDATE messages parsed: 26
Invalid or unparsed messages: 25
Report saved to: reports\bgp_vpls_mtu.json
{
  "bgp_updates": 26,
  "vpls_related_updates": 18,
  "advertised_mtu_values": [
    1500,
    9000
  ],
  "updates_with_parse_errors": 0
}
```

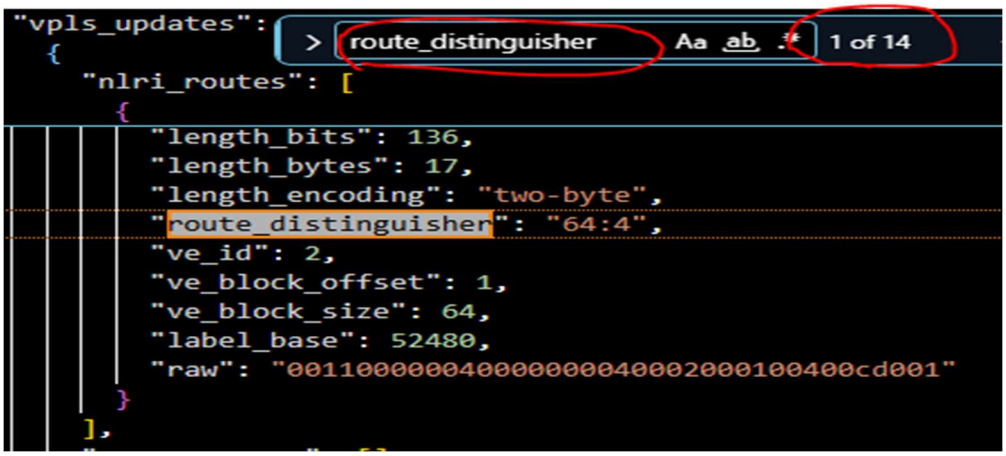
https://drive.google.com/drive/folders/1yWnTnv160Xu6QbzdhHlW8CrmA1HKno06?usp=sharing

**بررسی اطلاعات مربوط به VPLS درNLRI**

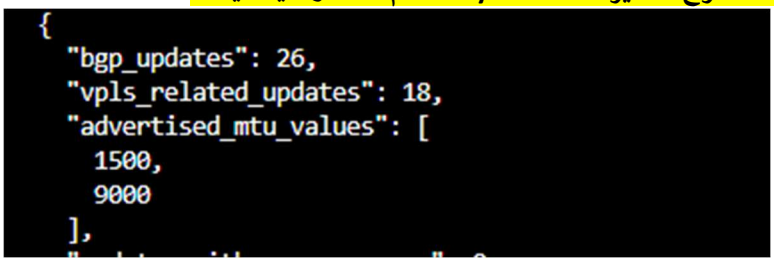


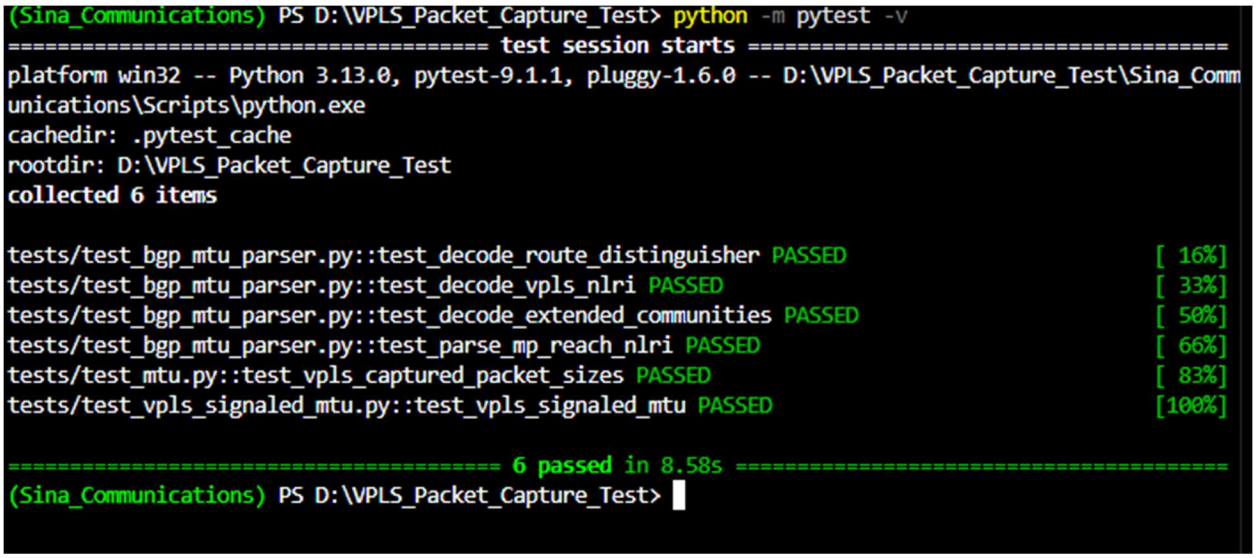
- استخراج اطلاعاتRoute Distinguisher .
- بررسیExtended Community های مرتبط باVPLS :




- استخراج مقادیر Layer-2 MTU اعلام‌شده در سیگنالینگ:




- ذخیره نتایج در قالبJSON .
- بررسی عملکرد بخش‌های مختلف برنامه با استفاده از تست‌های خودکار:



- **مدیریت فایل‌های پروژه و نسخه‌های کد باGit :**
- **آماده‌سازی ساختار پروژه برای نگهداری و توسعه درGitHub**

```
hamed@ccna MINGW64 /d/VPLS_Packet_Capture_Test (main)
$ git commit -m "chore: initialize repository, creating folders and fils + setting up the python environment"
[main (root-commit) 5671d32] chore: initialize repository, creating folders and fils + setting up the python environment
 9 files changed, 13 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 check_pcap.py
 create mode 100644 data/VPLS_PE(cisco)_PE(cisco).pcapng
 create mode 100644 doc/rfc-notes.md
 create mode 100644 readme.txt
 create mode 100644 report.txt
 create mode 100644 requirements.txt
 create mode 100644 src/vpls_mtu_checker/analyzer.py
 create mode 100644 tests/test_mtu.py

hamed@ccna MINGW64 /d/VPLS_Packet_Capture_Test (main)
$ git remote add origin https://github.com/HamedRabbani/vpls-mtu-validation-Sina-Communications-.git

hamed@ccna MINGW64 /d/VPLS_Packet_Capture_Test (main)
$ git push -u origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (10/10), 94.33 KiB | 4.49 MiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/HamedRabbani/vpls-mtu-validation-Sina-Communications-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

## ۳- آشنایی با مفاهیم استفاده‌شده:

### ۳.۱پروتکلBGP

پروتکل Border Gateway Protocol یا BGP برای تبادل اطلاعات مسیریابی بین سیستم‌های مستقل در شبکه استفاده می‌شود.

در این پروژه، تمرکز روی پیام‌های BGP UPDATE است؛ چون این پیام‌ها اطلاعات مربوط به مسیرها و ویژگی‌های آن‌ها را منتقل می‌کنند.

با بررسیAttribute های موجود در پیام‌های UPDATE می‌توان اطلاعات موردنیاز برای تحلیل سیگنالینگ VPLS را استخراج کرد.

### ۳.۲سرویسVPLS

سرویس Virtual Private LAN Service یا VPLS برای فراهم کردن ارتباط لایه دوم بین نقاط مختلف شبکه استفاده می‌شود.

در این پروژه، پیام‌های BGP مرتبط با VPLS بررسی می‌شوند تا اطلاعاتی مانندRoute Distinguisher ، اطلاعات NLRI و MTU اعلام‌شده استخراج شوند.

### ۳.۳مفهومMTU

پارامتر Maximum Transmission Unit یاMTU ، حداکثر اندازه بسته یا فریمی را مشخص می‌کند که یک رابط یا فناوری شبکه می‌تواند در شرایط تعریف‌شده منتقل کند.

در این پروژه، منظور از MTU استخراج‌شده، مقداری است که در اطلاعات سیگنالینگ مرتبط با Layer-2 MTU مشاهده می‌شود.

این مقدار لزوماً اثبات نمی‌کند که همان MTU در تمام مسیر شبکه قابل استفاده است؛ برای تأیید MTU عملیاتی، بررسی تنظیمات تجهیزات و انجام تست‌های مناسب نیز لازم است.

### ۳.۴فایل PCAP وPCAPNG

فایل‌های PCAP و PCAPNG برای ذخیره پکت‌های ثبت‌شده شبکه استفاده می‌شوند.

به‌جای بررسی دستی تمام پکت‌ها، برنامه این فایل‌ها را می‌خواند و اطلاعات موردنیاز را به‌صورت خودکار استخراج می‌کند.

### ۳.۵ Git وGitHub.

Gitیک ابزار Version Control است که برای مدیریت تغییرات فایل‌های پروژه استفاده می‌شود.

GitHubنیز بستری برای نگهداری مخزن کد، مشاهده تاریخچه تغییرات و به‌اشتراک‌گذاری پروژه است.

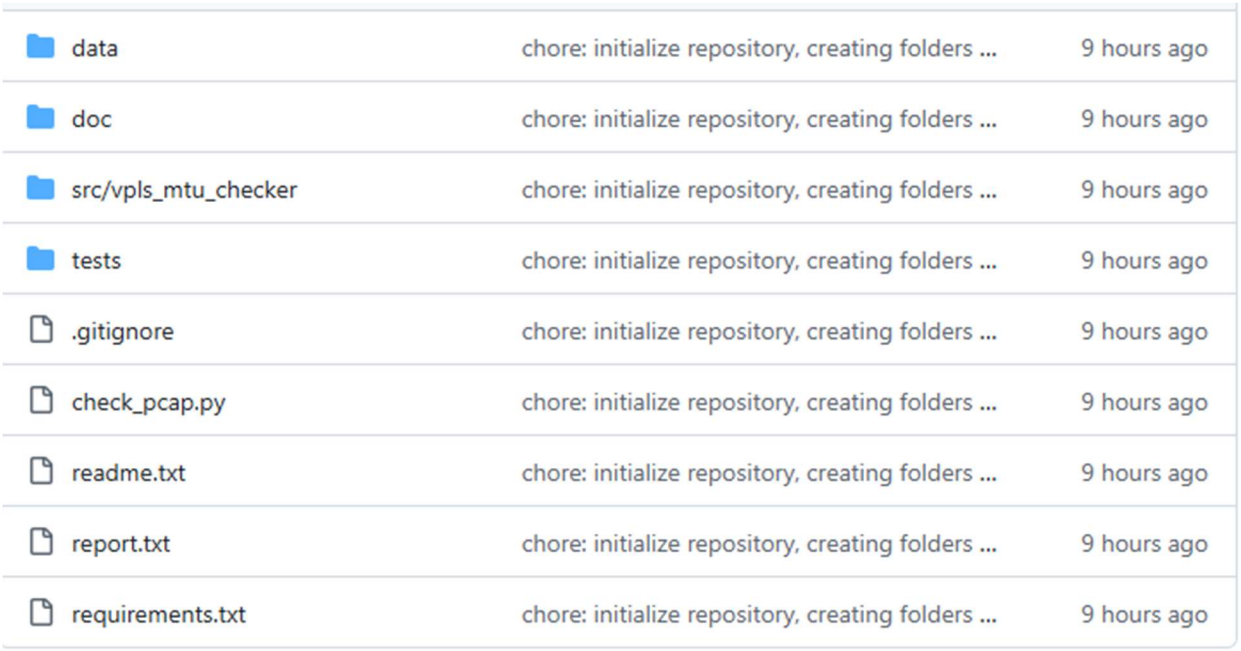
در این پروژه، Gitبرای ثبت تغییرات و GitHub برای نگهداری آنلاین مخزن پروژه استفاده می‌شود.

## ۴- ابزارها و فناوری‌های استفاده‌شده:

| کاربرد در پروژه | ابزار یا فناوری |
|---|---|
| پیاده‌سازی منطق تحلیل فایلCapture | Python |
| خواندن و بررسی پکت‌های شبکه | Scapy |
| اجرای تست‌های خودکار | pytest |
| فایل ورودی تحلیل شبکه | PCAPNG |
| ذخیره گزارش نهایی تحلیل | JSON |
| مدیریت نسخه‌های کد | Git |
| نگهداری و انتشار مخزن پروژه | GitHub |
| اجرای دستورهای پروژه در ویندوز | PowerShell |
| جداسازی وابستگی‌های پروژه از محیط اصلیPython | محیط مجازیPython |

## 5-ساختار پروژه:

ساختار فعلی پروژه بر اساس فایل‌های مشخص‌شده به شکل زیر است:




### ۵.۱پوشهdata

فایل Capture ورودی در این پوشه قرار می‌گیرد.

برنامه اطلاعات موجود در این فایل را می‌خواند و برای استخراج پیام‌های BGP و اطلاعات مرتبط با VPLS از آن استفاده می‌کند.

### ۵.۲پوشهscripts

کدهای اجرایی پروژه در این پوشه قرار دارند.

فایل inspect_bgp_mtu.pyوظیفه اصلی تحلیل Capture و تولید گزارش را بر عهده دارد.

فایل validate_nlri.pyبرای بررسی و اعتبارسنجی اطلاعات NLRI در نظر گرفته شده است. جزئیات دقیق عملکرد آن باید با نسخه فعلی کد تطبیق داده شود.

### ۵.۳پوشهtests

تست‌های خودکار پروژه در این پوشه قرار دارند.

این تست‌ها بخش‌هایی از منطق استخراج اطلاعاتBGP ، بررسی MTU و پردازش داده‌های مرتبط با VPLS را بررسی می‌کنند.

### ۵.۴ .پوشهreports

گزارش تولیدشده توسط برنامه در این پوشه ذخیره می‌شود.

نام فایل خروجی:

bgp_vpls_mtu.json

این فایل امکان بررسی نتایج تحلیل بدون نیاز به خواندن مستقیم تمام خروجی‌های ترمینال را فراهم می‌کند.

## ۶- مراحل پیاده‌سازی پروژه

### ۶.۱ .آماده‌سازی محیط توسعه

در مرحله اول، محیط Python برای اجرای پروژه آماده شد.

برای جلوگیری از تداخل کتابخانه‌های پروژه با سایر برنامه‌هایPython ، از محیط مجازی استفاده می‌شود.

دستورهای نمونه برای ایجاد و فعال‌سازی محیط مجازی در ویندوز:

python -m venv .venv

.\.venv\Scripts\Activate.ps1

در صورت وجود محیط مجازی از قبل، کافی است همان محیط فعال شود.

### ۶.۲ .خواندن فایلCapture

در این مرحله، برنامه فایل PCAPNG را با استفاده از Scapy می‌خواند.

اطلاعات پکت‌های موجود در فایل برای مراحل بعدی تحلیل استفاده می‌شوند.

تعداد پکت‌های خوانده‌شده یکی از شاخص‌های اولیه برای بررسی اجرای برنامه است.

### ۶.۳ .شناسایی جریان‌هایTCP

پیام‌های BGP در ارتباط TCP منتقل می‌شوند. به همین دلیل، برنامه باید داده‌های TCP را بررسی کند تا جریان‌های مرتبط با BGP را شناسایی کند.

در این مرحله، پکت‌های مربوط به جریان‌های شناسایی‌شده برای استخراج پیام‌های BGP آماده می‌شوند.

برای این بخش باید به محدودیت‌های بازسازی جریانTCP ، مانندRetransmission ، ترتیبSegment ها و وجود داده‌های ناقص، توجه کرد.

### ۶.۴ .استخراج پیام‌هایBGP

پس از آماده شدن جریان‌هایTCP ، برنامه داده‌ها را بررسی می‌کند تا پیام‌های BGP را از داخل جریان‌ها استخراج کند.

پیام‌های BGP انواع مختلفی دارند؛ بنابراین باید بین پیام‌های UPDATE و سایر انواع پیام‌ها تفاوت قائل شد.

پیام‌های غیر UPDATE لزوماً نامعتبر نیستند و نباید صرفاً به دلیل متفاوت بودن نوع آن‌ها خطا محسوب شوند.

### ۶.۵پردازش پیام‌هایUPDATE

در این مرحله، برنامه پیام‌های UPDATE را بررسی می‌کند.

اطلاعاتی مانندWithdrawn Routes ، Path Attributesو NLRI در ساختار پیام UPDATE قرار دارند.

برای استخراج اطلاعات موردنیاز پروژه، باید بخش Path Attributes و NLRI مطابق قالب پروتکل تفسیر شوند.

### ۶.۶استخراج اطلاعاتVPLS

در پیام‌های مربوط بهVPLS ، اطلاعات NLRI برای شناسایی مشخصات مسیر استفاده می‌شود.

در این پروژه، موارد زیر از اطلاعات قابل پردازش استخراج می‌شوند:

Route Distinguisher

شناسه VE یاVE ID

VE Block Offset

VE Block Size

Label Base

سایر اطلاعات پشتیبانی‌شده توسطParser

این اطلاعات برای بررسی محتوای سیگنالینگ VPLS استفاده می‌شوند.

### ۶.۷استخراجLayer-2 MTU

یکی از بخش‌های اصلی پروژه، بررسیExtended Community های پیام BGP و استخراج مقدار Layer-2 MTU است.

برنامه داده‌های مربوط به Extended Community را بررسی می‌کند و در صورت شناسایی قالب مورد انتظار، مقدار MTU را استخراج می‌کند.

در اجرای ثبت‌شده، دو مقدار MTU زیر گزارش شده‌اند:

1500

9000

این مقادیر نشان‌دهنده اطلاعات استخراج‌شده از Capture هستند و برای نتیجه‌گیری درباره MTU عملیاتی شبکه به بررسی‌های تکمیلی نیاز است.

### ۶.۸ .تولید گزارشJSON

در پایان تحلیل، نتایج در قالب JSON ذخیره می‌شوند.

فایل خروجی برای بررسی تعداد پیام‌ها، اطلاعاتVPLS ، مقادیر MTU و سایر داده‌های استخراج‌شده استفاده می‌شود.

مسیر گزارش:

reports/bgp_vpls_mtu.json

## ۷- تست و بررسی صحت برنامه:

برای بررسی بخش‌های مختلف برنامه، تست‌های خودکار با استفاده از pytest اجرا شده‌اند.

هدف از تست‌ها، بررسی رفتار توابع و اطمینان از عملکرد صحیح بخش‌هایی از منطق پردازش داده‌هاست.

برای اجرای تست‌ها، دستور زیر استفاده می‌شود:

python -m pytest -v

در یکی از اجراهای ثبت‌شده، نتیجه زیر به دست آمده است:

۶ تست موفق.

۷.۱بررسی پیام‌های پردازش‌نشده

**در یکی از اجراهااین پیام نیز نمایش داده شده است:**

**Invalid or unparsed messages: 25**

**برای مشخص شدن علت، لازم است نوع پیام‌های BGP جداگانه شمارش شود.**

اگر بخشی از پیام‌ها از نوع OPEN یا KEEPALIVE باشند، نباید صرفاً به دلیل اینکه UPDATE نیستند، باید به‌عنوان UPDATE نامعتبر شمرده شوند.

از طرف دیگه اگر پیام‌های UPDATE واقعاً پردازش نشده باشند باید علت آن مشخص شود.

همچنین باید قالب طول NLRI در Capture واقعی با چیزی که Parser انتظار دارد تطبیق داده شود.

## ۸- نتایج اولیه پروژه:

| شاخص | نتیجه ثبت‌شده |
|---|---|
| تعداد پکت‌های خوانده‌شده | 4,897 |
| پیام‌های BGP UPDATE پردازش‌شده | 26 |
| پیام‌های مرتبط باVPLS | 18 |
| مقادیر MTU استخراج‌شده | 1500و 9000 |
| تست‌های موفق | 6 |

## ۹- مراحل مدیریت پروژه باGit :

برای مدیریت تغییرات کد و آماده‌سازی پروژه جهت نگهداری درGitHub ، از Git استفاده می‌شود.

این بخش باید مطابق کارهایی که واقعاً انجام شده‌اند تکمیل شود. دستورهای زیر مراحل متداول این فرایند را نشان می‌دهند.

۹.۱ .بررسی وضعیت مخزن

ابتدا وضعیت مخزن با دستور زیر بررسی می‌شود:

git status

این دستور نشان می‌دهد کدام فایل‌ها تغییر کرده‌اند، کدام فایل‌ها هنوز Track نشده‌اند و آیا تغییراتی برای Commit وجود دارد یا نه.

## ۹.۲ .تنظیم فایلgitignore.

فایل gitignore.برای مشخص کردن فایل‌ها و پوشه‌هایی استفاده می‌شود که نباید وارد مخزن Git شوند.

برای این پروژه، مواردی مانند محیط مجازی، فایل‌های موقت Python و فایل‌های محلی تنظیمات باید بررسی شوند.

نمونه:

.venv/

venv/

__pycache__/

*.py[cod]

.pytest_cache/

.env

.idea/

.vscode/

فایل Capture و گزارش JSON نیز باید آگاهانه مدیریت شوند. اگر فایل Capture حجیم یا دارای اطلاعات حساس شبکه باشد، بهتر است بدون بررسی مجوز انتشار وارد مخزن عمومی نشود.

## ۹.۳بررسی فایل‌های آماده ثبت

پس از تنظیم gitignore.، دوباره وضعیت مخزن بررسی می‌شود:

**git status**

در این مرحله باید بررسی شود که فایل‌های غیرضروری و اطلاعات حساس برای Commit انتخاب نشده باشند.

## ۹.۴ .افزودن فایل‌ها بهStaging Area

برای آماده‌سازی تغییرات جهت ثبت، از دستور زیر استفاده می‌شود:

**git add .**

این دستور تغییرات مجاز را به Staging Area اضافه می‌کند.

قبل ازCommit ، بهتر است محتوای فایل‌های آماده ثبت بررسی شود:

**git status**

**git diff --cached**

## ۹.۵ثبت اولینCommit

پس از بررسی تغییرات، می‌توان آن‌ها را در تاریخچه Git ثبت کرد:

**git commit -m "build: initialize VPLS packet capture analysis"**

پیام Commit مشخص می‌کند که این مرحله مربوط به آماده‌سازی اولیه پروژه است.

اگر Commit قبلاً ثبت شده، نباید صرفاً برای گزارش دوباره آن را اجرا کرد. در این حالت، تاریخچه با دستور زیر بررسی می‌شود:

**git log --oneline**

### ۹.۶اتصال به مخزنGitHub

اگر مخزن GitHub قبلاً ساخته شده و هنوز به پروژه محلی متصل نیست، آدرس واقعی مخزن باید تنظیم شود.

نمونه دستورها:

**git remote add origin <GITHUB_REPOSITORY_URL>**

**git remote -v**

اگر Remote از قبل وجود دارد، ابتدا با git remote -vآن را بررسی کرد و دستور git remote addرا دوباره اجرا کرد.

### ۹.۷ارسال تغییرات بهGitHub

پس از آماده شدن Commit و اتصال صحیحRemote ، تغییرات به GitHub ارسال می‌شوند.

در صورتی که شاخه اصلی پروژه mainباشد:

**git push -u origin main**

این دستورCommit های محلی را به مخزن Remote ارسال می‌کند.

اگر نام شاخه اصلی متفاوت است، باید از نام واقعی آن استفاده شود.

### ۹.۸بررسی نهایی مخزن

در پایان، وضعیت محلی پروژه و فایل‌های مخزن GitHub بررسی می‌شود.

دستور:

**git status**

اگر تغییر ثبت‌نشده‌ای وجود نداشته باشد، معمولاً پیام زیر نمایش داده می‌شود:

**nothing to commit, working tree clean**

این پیام نشان می‌دهد در آن لحظه تغییر ثبت‌نشده‌ای در Working Tree وجود ندارد؛ اما به‌تنهایی اثبات نمی‌کند که تمام فایل‌های موردنیاز در GitHub قرار گرفته‌اند

## ۱۰- محدودیت‌های پروژه:

با وجود پیاده‌سازی بخش اصلی تحلیل، موارد زیر باید در نظر گرفته شوند:

- برنامه اطلاعات موجود در فایل Capture را بررسی می‌کند و نمی‌تواند اطلاعاتی را که در فایل ثبت نشده‌اند استخراج کند.
- استخراج مقدار Layer-2 MTU از سیگنالینگBGP ، به‌تنهایی ثابت نمی‌کند که همان MTU در عمل روی تجهیزات یا در تمام مسیر شبکه قابل استفاده است.
- صحت نتیجه به درستی بازسازی جریان TCP و تفسیر ساختار پیام‌های BGP وابسته است.
- تست‌های واحد موفق، لزوماً تضمین نمی‌کنند که تمام پیام‌های فایل واقعی به‌درستی پردازش شده‌اند.
- تعداد پیام‌های پردازش‌نشده و قالب طول NLRI در Capture واقعی باید پیش از نهایی کردن نتیجه بررسی شوند.
- برای تأیید عملکرد عملیاتیVPLS ، بررسی تنظیمات تجهیزات و انجام تست‌های شبکه متناسب با محیط نیز لازم است.

## ۱۱- دستاوردهای پروژه:

در این پروژه، مراحل اولیه ساخت یک ابزار برای تحلیل فایل Capture و استخراج اطلاعات مرتبط با BGP VPLS انجام شده است.

دستاوردهای فعلی عبارت‌اند از:

پیاده‌سازی اولیه تحلیل فایل Capture باPython

استفاده از Scapy برای خواندن پکت‌های شبکه

استخراج پیام‌های BGP و پردازشUPDATE ها

پیاده‌سازی بخش‌هایی برای استخراج اطلاعات VPLS وLayer-2 MTU

ذخیره نتایج در قالبJSON

ایجاد تست‌های خودکار باpytest

استفاده از Git برای مدیریت تغییرات کد

آماده‌سازی مستندات پروژه برای ارائه و نگهداری

مواردی که هنوز نیاز به تأیید دارند، باید پیش از ارائه نهایی بررسی شوند.

## ۱۲- جمع‌بندی نهایی:

هدف این پروژه، بررسی سیگنالینگ BGP مرتبط با VPLS و استخراج اطلاعات MTU از فایل Capture بوده است.

برای این منظور، برنامه‌ای با Python توسعه داده شده که فایل PCAPNG را می‌خواند، پیام‌های BGP را استخراج می‌کند، اطلاعات مرتبط با VPLS را بررسی می‌کند و نتیجه را در قالب JSON ذخیره می‌کند.

در یکی از اجراهای قبلی، 4,897 پکت خوانده شده، 26 پیام BGP UPDATE پردازش شده و 18 پیام مرتبط با VPLS گزارش شده است. همچنین مقادیر MTU برابر با 1500 و 9000 استخراج شده‌اند و 6 تست خودکار با موفقیت اجرا شده‌اند.

این نتایج اولیه نشان می‌دهند که مسیر اصلی تحلیل پیاده‌سازی شده است؛ با این حال، برای ارائه نتیجه نهایی باید خروجی آخرین اجرا، شمارش پیام‌های نامعتبر و تفسیر NLRI در فایل واقعی بررسی شوند.

در کنار توسعه برنامه استفاده از Git و GitHub نیز بخشی از فرایند مدیریت کد و مستندسازی پروژه است. ثبت صحیح مراحل کار و ارائه اسکرین‌شات‌های واقعی از اجرای برنامه، تست‌ها و تاریخچه Git می‌تواند روند توسعه پروژه را شفاف‌تر کند.